



Dominando o CSS Grid Layout

Do zero ao seu primeiro layout bidimensional.

Objetivos da aula: Entender quando usar Grid, dominar as unidades de medida e criar um layout completo.

Flexbox vs. CSS Grid: Qual a diferença?

Característica	Flexbox	CSS Grid
Dimensão	Unidimensional (1D)	Bidimensional (2D)
Direção	Trabalha em linhas OU colunas	Trabalha em linhas E colunas simultaneamente
Foco	Focado no conteúdo	Focado no layout macro

Quando usar cada um?

Flexbox para menus/botões; Grid para estruturas de páginas/dashboards.

Anatomia do Grid (Conceitos Essenciais)

Container (Pai) e Items (Filhos)

O elemento pai com `display: grid` é o ***Container***. Os elementos diretamente dentro dele são os ***Items*** (Filhos).

Componentes do Grid

- **Grid Lines:** linhas invisíveis de grade
- **Grid Tracks:** as colunas/linhas em si
- **Grid Cell:** uma caixa
- **Grid Area:** grupo de células

Unidades de Medida: fr, px e auto

px (Pixels)

Tamanho fixo e rígido.

auto

Adapta-se ao tamanho do conteúdo do elemento.

fr (Fractional Unit)

A mágica do Grid! Divide o espaço disponível proporcionalmente.

Exemplo prático: Comparativo visual

Exemplo 1: 100px auto 1fr

```
grid-template-columns: 100px auto 1fr;
```

A primeira coluna tem 100px fixos. A segunda se adapta ao conteúdo. A terceira ocupa o restante do espaço disponível.

Exemplo 2: 1fr 2fr 1fr

```
grid-template-columns: 1fr 2fr 1fr;
```

O espaço é dividido em 4 partes: a primeira e a terceira coluna recebem 1 parte cada, e a do meio recebe 2 partes.

Definindo a Estrutura: grid-template-columns e grid-template-rows

Como ativar o grid:

```
display: grid;
```

grid-template-columns

Define o número e o tamanho das colunas do grid. Cada valor separado por espaço representa uma coluna.

```
grid-template-columns: 200px 1fr 1fr;
```

grid-template-rows

Define o número e o tamanho das linhas do grid. Cada valor separado por espaço representa uma linha.

```
grid-template-rows: 80px 1fr 60px;
```

Dica de atalho útil: Uso da função repeat()

Use a função `repeat()` para evitar repetição de código. Por exemplo: `repeat(3, 1fr)` é equivalente a escrever `1fr 1fr 1fr`.

```
grid-template-columns: repeat(3, 1fr);
```



Propriedade gap

Como espaçar os itens sem precisar de margin.

column-gap

```
column-gap: 20px;
```

Define o espaço entre as colunas do grid.

row-gap

```
row-gap: 16px;
```

Define o espaço entre as linhas do grid.

gap (atalho)

```
gap: 16px 20px;
```

Atalho que define row-gap e column-gap em uma única declaração.

Criando Layouts Visuais com grid-template-areas

O conceito de "desenhar o layout com texto".

Como nomear áreas no container pai usando aspas e nomes legíveis (header, sidebar, main, footer).

```
grid-template-areas:
```

```
"header header"
```

```
"sidebar main "
```

```
"footer footer";
```

O uso do ponto (.) para deixar espaços vazios.

```
grid-template-areas:
```

```
"header header"
```

```
". main "
```

```
"footer footer";
```

Conectando os Filhos com grid-area

Como aplicar a propriedade `grid-area` nos elementos filhos para encaixá-los nas áreas desenhadas pelo pai.

Exemplo de código CSS completo do Pai e dos Filhos lado a lado.

Container (Pai)

```
.container {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: 80px 1fr 60px;  
  grid-template-areas:  
    "header header"  
    "sidebar main "  
    "footer footer";  
  gap: 16px;  
}
```

Elementos (Filhos)

```
.header {  
  grid-area: header;  
}  
  
.sidebar {  
  grid-area: sidebar;  
}  
  
.main {  
  grid-area: main;  
}  
  
.footer {  
  grid-area: footer;  
}
```


Posicionamento Avançado: Grid Lines e span

As **Grid Lines** são as linhas invisíveis numeradas que cortam o layout, definindo o esqueleto do seu grid. Um grid de 3 colunas, por exemplo, possui 4 linhas verticais.

1

Início e Fim Explícito

Posicione um item especificando as linhas de grade de início e fim para colunas ou linhas.

```
grid-column: 1 / 3;
```

(Começa na linha 1 e termina na linha 3)

2

O Poder do span

A palavra-chave `span` permite que um item ocupe um número específico de faixas, funcionando como "mesclar células".

```
grid-column: span 2;
```

(Ocupa 2 colunas de largura)

Exemplo Prático: Destaque de Conteúdo

Imagine um card de notícias, um banner promocional ou qualquer elemento que precisa de mais visibilidade. Com o CSS Grid, podemos fazê-lo ocupar mais espaço de forma elegante, usando `span` para estender sua largura e altura através de múltiplas células.

Definindo o Destaque com `span`:

```
.card-principal {  
  grid-column: 1 / span 2; /* Ocupa 2 colunas, começando da linha de grade 1 */  
  grid-row: 1 / span 2; /* Ocupa 2 linhas, começando da linha de grade 1 */  
}
```

Neste exemplo, o elemento com a classe `.card-principal` se posiciona a partir da primeira linha de grade vertical e se estende por duas colunas. Simultaneamente, ele também começa na primeira linha de grade horizontal e se estende por duas linhas, criando um bloco de 2x2 células no grid, ideal para um conteúdo que merece destaque.

Resumo

Checklist do que aprendemos hoje.

01

Diferença entre Flexbox (1D) e CSS Grid (2D)

03

Unidades de medida: `px`, `auto` e `fr`

05

Função `repeat()` para evitar repetição

07

Criação de layouts com `grid-template-areas`

09

Posicionamento com Grid Lines: `grid-column` e `grid-row` com início e fim explícito

02

Anatomia do Grid: Container, Items, Lines, Tracks, Cell e Area

04

Propriedades `grid-template-columns` e `grid-template-rows`

06

Propriedade `gap` para espaçamento

08

Conexão dos filhos com `grid-area`

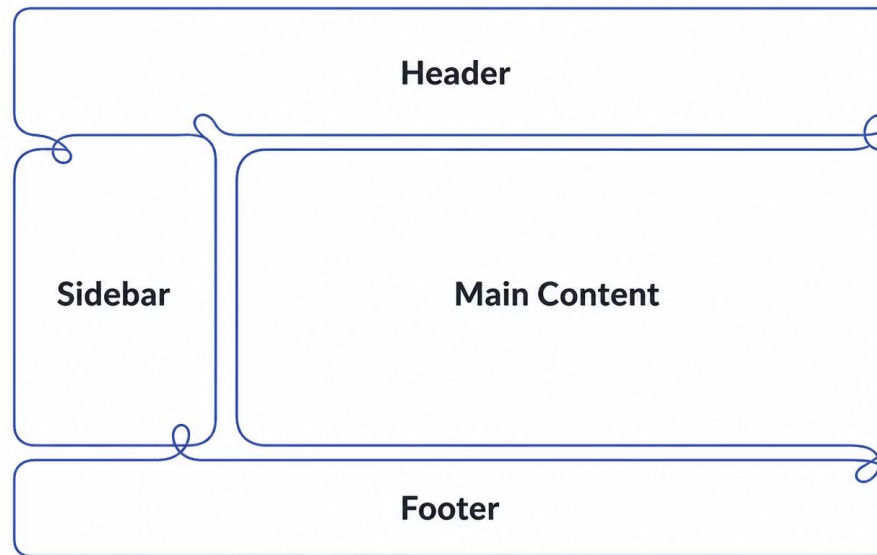
10

Palavra-chave `span` para ocupar múltiplas colunas ou linhas

DESAFIO PRÁTICO

Desafio Prático

Seu desafio é criar o layout clássico de um blog, que consiste em Header, Sidebar, Conteúdo Principal e Footer, utilizando o poder do `grid-template-areas`.



Aplique os conceitos de `grid-template-columns`, `grid-template-rows` e `gap` para estruturar e espaçar os elementos de forma coesa e responsiva.